

Day 13: Core Python Automation Notebook

Binomial & Poisson Distributions — Pure Manual and Scipy Shortcut Architecture

Welcome to the Python Programming Edition of Day 13 Core Analytics. This notebook contains the full manual and shortcut source codes for both Binomial and Poisson frameworks along with deep line-by-line execution logic.

Day 13 Core Analytics ke Python Programming Notebook me aapka swagat hai. Is document me Binomial aur Poisson dono frameworks ke manual aur shortcut source codes ko unke poore internal execution routing aur logic ke sath bitha diya gaya hai.

1. Binomial Distribution: Pure Manual Engine Code

This code implements the manual multi-engine logic for the Ring Factory numerical problem ($n=5, k=2, p=0.10$) without utilizing pre-built data science libraries.

Yeh code bina kisi library ke sateek humare notes wale Ring Factory Problem ($n=5, k=2, p=0.10$) ke upar poore manual mathematical steps ko replicate karta hai.

```
def calculate_factorial(number):
    # ENGINE 1: Factorial Engine (n!) - Loop se factorial nikalta hai
    if number == 0 or number == 1:
        return 1
    result = 1
    for i in range(2, number + 1):
        result *= i
    return result

def find_total_conditions(n, k):
    # ENGINE 2: Combination Matrix Engine (nCk) = n! / (k! * (n-k)!)
    numerator = calculate_factorial(n)
    denominator = calculate_factorial(k) * calculate_factorial(n - k)
    return numerator // denominator

if __name__ == "__main__":
    n, k, p = 5, 2, 0.10
    q = 1 - p

    total_paths = find_total_conditions(n, k) # Returns 10 Raaste
    single_chain_prob = (p ** k) * (q ** (n - k)) # Returns 0.00729
    real_result = total_paths * single_chain_prob

    print(f"Result: {real_result:.4f}") # Output: 0.0729 (7.29%)
```

🧠 Line-By-Line Execution & Traffic Routing (Binomial Manual):

- **Entry Gate:** Python execution sabsbe pehle `if __name__ == "__main__":` ke andar se shuru karta hai aur parameters load karta hai.
- **First Jump:** Jab control `find_total_conditions(5, 2)` par aata hai, toh processor main code ko pause karke combination engine par jump maarta hai.
- **Deepest Factorial Execution:** Combination engine ke andar baithi line `calculate_factorial(n)` control ko sabsbe upar loop par le jaati hai jahan $5!$ (120) nikalta hai. Wahi se $2!$ aur $3!$ nikal kar data transfer hota hai.
- **Final Return:** Pura system calculate hokar $120 // 12 = 10$ data return karta hai aur main gate par aa kar final multiply complete karta hai.

2. Binomial Distribution: Data Science Shortcut Code

Using the industry standard `scipy.stats` module to condense the entire calculation engine down to a single optimized execution call.

Industry standard Scipy library ka use karke poore automation framework ko sirf ek single line module me collapse kiya gaya hai.

```
# Scipy library se Binomial Core Probability engine ko call kiya
from scipy.stats import binom

n, k, p = 5, 2, 0.10

# [CRITICAL ATTENTION]: Scipy me parameter order perfectly (k, n, p) hota hai!
# Background Mechanics: A)  $5C2 = 10$  paths, B) Powers Multiply, C) Final Product
real_result = binom.pmf(k, n, p)

print(f"🏆 SCIPY RESULT: {real_result:.4f} ({real_result*100:.2f}%)")
```

What is binom.pmf?

Full Form: Binomial Probability Mass Function.

Internal Mechanics: Discrete counting data (jaise exact 2 pieces, no decimals) ka chance nikalne ke liye yeh automatically back-end me combination, math powers aur target matrix ko aapas me integrate kar deta hai bina koi extra code likhe.

3. Poisson Distribution: Pure Manual Engine Code

Implementing the manual formulation for the continuous Showroom Footfall problem ($\lambda = 3$, $k = 4$) focusing on Euler's constant decay execution.

Showroom Footfall problem ($\lambda = 3, k = 4$) ke liye manual core equation setup, jahan e ki power ko math rules ke mutabik resolve kiya gaya hai.

```
def calculate_factorial(number):
    if number == 0 or number == 1: return 1
    res = 1
    for i in range(2, number + 1): res *= i
    return res

def calculate_exponential_decay(lambda_val):
    # Manually solves e^-lambda as 1 / (e^lambda)
    e = 2.71828
    return 1 / (e ** lambda_val)

if __name__ == "__main__":
    lambda_val, k = 3, 4

    umeed_engine = lambda_val ** k # 3^4 = 81 (Expectation Matrix)
    kudrat_balance = calculate_exponential_decay(lambda_val) # e^-3 ≈ 0.049787
    chaos_controller = calculate_factorial(k) # 4! = 24 (Normalizer)

    real_result = (umeed_engine * kudrat_balance) / chaos_controller
    print(f"Poisson Manual Result: {real_result:.4f}") # 0.1680 (16.80%)
```

4. Poisson Distribution: Data Science Shortcut Code

Collapsing the continuous time framework down into the highly efficient `poisson.pmf` scipy library structure.

Time aur Space ke is continuous distribution calculation ko Scipy library ke sabse compact design pattern me convert kiya gaya hai.

```
from scipy.stats import poisson

lambda_val, k = 3, 4

# [CRITICAL ATTENTION]: Scipy call format -> poisson.pmf(k, mu) jahan mu = lambda_val hai
real_result = poisson.pmf(k, lambda_val)

print(f"🏆 SCIPY POISSON RESULT: {real_result:.4f} ({real_result*100:.2f}%)")
```

The Infinite Nanoseconds Connection:

As proved mathematically during our core logic discussions, the shortcut code executing `poisson.pmf` encapsulates the full transformation of a continuous spectrum where artificial time intervals (n^k) balance out perfectly across the numerator matrix and denominator paths, removing time structural constraints.

Core Synthesis Review: `poisson.pmf` back-end me bina infinite nanoseconds (n^k) ke jhanjhat me pade, seedhe umeed (λ^k) aur decay function ($e^{-\lambda}$) ko chaos controller se divide karke sateek outcome generate kar deta hai.

● End of Day 13 Integrated Automation Notes ● Processed Perfectly ● Abu Fraz Abbas ---